

Building one shared dictionary of what moves stocks.

This walks the **whole machine**, one idea per slide — nothing left out. First **the data model** (what we're building), then **how we build it**, then **how it gets used live**. The **Appendix (slides 42-57)** explains every code file, one at a time. Open the `Details` boxes for the deeper version.

How to read this

1. **Use → and ←** (or the buttons at the bottom) to move between slides.
2. **Each slide = one concept.** The big text is the whole point; panels are the detail.
3. **Click a `Details` box** for the deeper / technical version — that's where the fine print lives.
4. **Examples are real** — pulled from an actual run on the Restaurants industry.

02 The goal (one analogy)

03 Why it's built this way

04 The one golden rule

05 The 3 layers (the model)

06 The graph shape (the edges)

07 Who creates what

08 The Driver node (the label)

09 fact_type — the 4 kinds

10 The big picture (the tree)

11 The record (build output)

12 The two kinds of files

13 Stage 1 — the Leaf

14 Resolve

15 Fetch

16 Chunk

17 The blind bots

18 Converge

19 Stage 2 — Reconcile

20 Guard & slice

21 Two judges

22 The skeptic

23 The same-name check

24 The writer (code)

25 The validator (gate)

26 Final steps: fact_type + links

27 Stage 3 — The Fold

28 Fold mechanics

29 Orchestrate, repair, resume

30 Models & cost

31 The DriverUpdate (the fact)

32 driver_state — the 4 lanes

33 The number layer

34 Number rules + examples

35 The verdict
(EXPLAINED_BY)

36 Identity & change-
tracking

37 Invariants &
persistence

38 Using it live (later)

39 The guidance
bridge (idea)

40 Where each part
stands

41 Safety recap &
status

A Appendix: every file,
one by one

▼ Tiny glossary (the words you'll see — skim now, refer back later)

driver	a real-world cause that moves a stock (oil price, store traffic). The reusable label — a "class".
DriverUpdate	one real fact about a driver on one event ("same-store sales rose 3%"). The instance .
verdict	a producer's judgement that a fact <i>caused</i> a stock move. Lives on a link (<code>EXPLAINED_BY</code>), not a box.
fact_type	a driver's permanent KIND: <code>metric</code> / <code>guidance</code> / <code>surprise</code> / <code>action_event</code> .
driver_state	what happened this event (rose, beat, suspended...) — picked from a fixed list per <code>fact_type</code> .
producer	the AI that reads a real event and writes the facts: <code>earnings-learner</code> OR <code>news-driver</code> .
catalog	the final, cleaned dictionary of all driver labels (the classes).
seed	the raw, messy first draft of the catalog, before cleaning.
mention vs fact	a mention is a quote that proves a NAME is worth having; a fact is a real event-level thing a producer judged. A mention never makes a DriverUpdate.
SAME_AS	a reversible link saying two driver names are the same cause (both survive, never deleted).
fail-closed	when unsure, do the safe thing (keep separate) — never guess a merge.
PIT	"point-in-time": never show the AI a name, number, or outcome from the future.
KPI / fiscal.ai	a metric the company itself reports (e.g. "same-store sales"), pulled from the fiscal.ai data feed. It can justify a NAME, but it's not a real event — so it never makes a fact.

Think of it as one master ingredient list — for the whole stock market.

We want **one list of every cause that moves a stock**, where the same cause always has the same name. So when "beef price" jumps, we instantly know **every** company it touches → a trade signal.

The restaurant analogy

Imagine every restaurant in the country writes its own menu, separately:

- Restaurant A writes `guest count` . Restaurant B writes `customer transactions` .
- Those are the *same thing* → they must collapse to **one** name.
- But `delivery mix` at a restaurant $\neq$ `delivery mix` at a shipping company → must stay **two** names.

The whole project is: collect every "menu," then carefully decide what is truly the same and what isn't — without ever guessing.

Why same-name matters

If two companies use one shared name for a cause, software can watch that one name and detect a change across all of them at once. That's the payoff: **learn once, predict everywhere.**

Why it must be tiny

Keep the list as small as possible while still being specific. No bloat. But "small" never wins over "correct" — we never merge two different causes just to shrink the list.

▼ The one promise that makes it tradeable: one name = one meaning

A single driver name must **never** stand for two different real-world causes, anywhere, at any level. If it ever does, software would read a fake connection and place a bad trade. So the entire build is bent toward protecting that one promise — and when two names truly mean the same, they're linked with a *reversible* link (never a destructive delete), so the originals can always be recovered.

Two earlier tries failed. Every rule here exists to avoid their deaths.

This isn't over-engineering — it's **two specific funerals** we refuse to repeat.

Attempt	How it died	The fix baked in now
v1	Used a fixed, pre-written word-list. Real names didn't fit it → it rejected 82% of them.	Open vocabulary — names come straight from the source text, never a fixed list.
v2	Merged too eagerly. Many different ideas collapsed into one mushy <code>revenue_demand</code> .	Fail-closed merging — only merge on <i>exact</i> same meaning; a skeptic attacks every merge; when unsure, keep separate.

The one mistake we fear most

A **wrong merge**. If we wrongly say two companies share a driver, software reads a false connection between them and places a **bad trade**. So merging is the dangerous direction — and the entire design is built to make wrong merges hard.

▼ The two judgment moments that caused both failures

- "Is this already named?" → get it wrong and you fragment one idea into many names.
- "Is this a real, specific-enough driver?" → get it wrong and you drift into over-broad mush.

Rules can *narrow* these two judgments but never remove them. So the design keeps the judgment with AI, wraps it in skeptics, and lets *code* catch every structural mistake.

One rule explains the entire system.

AI decides meaning. Code does structure. Every merge must survive a skeptic — or it doesn't happen.

AI (the agents)

Anything that needs *understanding*:

- "Is this a real driver?"
- "Do these two mean the same?"
- "Try to break this merge."

Code (Python / JS)

Anything *mechanical*:

- Pull data, split text, count.
- **Write the final file.**
- Check everything & hard-stop on any flaw.

Why this split is the whole trick

Because **code** writes the catalog, an AI **cannot sneak in a merge** the skeptic never approved. The old design let an AI write the file — so it could quietly re-create the v2 mush. Now that's structurally impossible. The AI only hands code a short list of approved decisions.

fail-closed

When in doubt, don't merge. A missing or uncertain answer is treated as "no." The safe direction is always *keep separate*.

Three things, not one: a *label*, a *fact*, and a *verdict*.

Before any code, picture the end result. The whole system stores exactly **three** kinds of thing. Get these three and everything else clicks.

1 · Driver

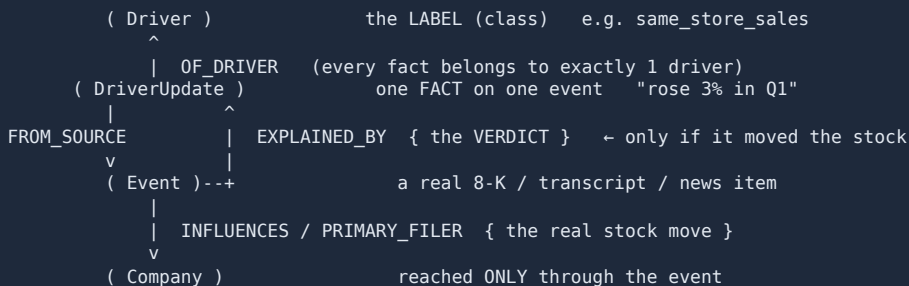
The reusable **label** (a "class"). Just a clean name + its permanent kind. `same store sales`. Built **once**.

2 · DriverUpdate node

One **fact** on one event (an "instance"): *"this driver did X on this day."* Many pile up under one label over time.

3 • Verdict edge

A judgement that the fact **caused** the stock move.
It is **not a box** — it's a labelled *link*
(EXPLAINED BY). A fact can exist with no verdict.



The one analogy

A **Driver** is a blank *thermometer* (`oil_price`). A **DriverUpdate** is one *reading* taken at a real event ("oil rose to \$80"). The **verdict** is a separate note clipped onto the event: *"and that reading is why the stock fell."* Three separate things — never mashed into one.

▼ Why is the verdict an *edge* and not its own box?

Because a verdict only means something **against the real return** — and the return lives on the Event → Company link. A company has hundreds of thousands of returns, so it can't say *which* move a verdict is about; the **event** can. So the verdict hangs off the event, one hop from its grade. (It also means a market-wide driver like oil_price needs no special case — no company is stored on the driver layer at all.)

Five links wire it together — and two of them point opposite ways.

Every connection in the graph is one of just five links. The trick to remember: a fact and its event are joined by **two** separate one-way links, never one.

Link	From → To	Means	When
OF_DRIVER	DriverUpdate → Driver	this fact is an instance of this driver	always (exactly 1)
FROM_SOURCE	DriverUpdate → Event	this fact came from this event (its evidence)	always
EXPLAINED_BY	Event → DriverUpdate	this event's move is explained by this fact the verdict	0, or 1 per producer
SAME_AS	Driver → Driver	two names are the same cause (reversible)	only on exact-same meaning
INFLUENCES / PRIMARY_FILER	Event → Company	the real stock move already exists	existing pipeline

The key subtlety: two opposite links between a fact and its event

FROM_SOURCE points **down** (fact → event): "I came from here." **EXPLAINED_BY** points **up** (event → fact): "you explain my move." They are **separate** because a fact can be *reported* in an event without *explaining* its move. Never collapse them into one link.

▼ Three more rules baked into the shape

- **No company on the driver layer.** The company is reached only via **Event → Company** (verified: every event has exactly one company). A market-wide driver takes the exact same shape.
- **No BASED_ON link.** Trends are *queried* (each fact's state + numbers carry the value), never stored as links between facts.
- **Same fact in two different events = two DriverUpdates** (keyed by event). They're only combined in a read-time *view*, never merged in storage.
- **If two producers ever judge one event** → two parallel **EXPLAINED_BY** links, one per producer (they can't collide).

This mirrors the production **Guidance** graph (a class node + per-event instances), already live at **548 Guidance + 8,432 GuidanceUpdate** nodes — so the shape is battle-tested.

The build makes *labels*. A producer makes *facts*. Nothing else.

This is the single most important rule of the whole design. Two jobs, two owners — and they never trade places.

Catalog build → the CLASS

In **one run**: read non-news sources + the company's own reported metrics (**KPIs**) → coin/reuse driver **names** → as final steps stamp each driver's `fact_type` + optional links.

It never creates a DriverUpdate.

Producer → the FACTS

An AI reading a **real event** (`earnings-learner` or `news-driver`) decides the event-level fact and writes the **DriverUpdate** (+ the verdict, if it moved the stock).

Producers are the ONLY makers of DriverUpdates — live or backfill.

Mention vs fact — the line that splits the two jobs

```
build sees: "...comparable restaurant sales trends..." → a MENTION → justifies a NAME
producer sees: "comp sales ROSE 3% in Q1 vs +1% guided" → a FACT → writes a DriverUpdate
```

A bare mention never becomes a DriverUpdate. The producer's **gate** drops it. (Generic risk boilerplate — "litigation could harm us" — is a mention, so it's dropped too.)

Three things that are deliberately forbidden

- **No build-time "seeder."** A second thing that invents facts would be a rival source of truth (it lacks the producer's prior-period memory + consensus) → silent errors. Forbidden.
- **KPI-only evidence makes a NAME only** — never a DriverUpdate (a KPI has no real event, date, or move).
- **No "needs >2 events" rule.** A single event is a valid fact (a CEO resignation, an FDA approval, a guidance cut). "Seen often" is a read-time view, never a write gate.

Want early history? Run the producer over the past (a "backfill")

That's the **only** correct way to "seed" — same producer, same judgement, one source of truth, zero duplicate logic. A backfilled fact is just flagged `produced_mode = backfill` so we know it could have used hindsight.

▼ The 8 locked points + the tensions we already settled (so they're never reopened)

- **1.** Build creates the Driver CLASS (name + fact_type + optional links) in one run, never a DriverUpdate.
- **2.** A DriverUpdate = a real event-level FACT (state / change / surprise / guidance / action) — not a mention. *(Supersedes the older "= a change" wording: an action like a CEO exit has no X→Y change but is still a fact.)*
- **3.** Producers are the only creators of DriverUpdates (live + backfill).
- **4.** KPI-only evidence → a name only.
- **5.** No separate build-time seeder / change-detector (it would be a rival source of truth).
- **6.** No ">2 events" write rule — a single event can be a fact; recurrence is read-time only.
- **7.** Early history = run the producer as a backfill.
- **8.** The verdict (EXPLAINED_BY) is separate: a fact may exist with no verdict.

Tensions logged & closed: "the build already sees the quote → seed a fact for free" → No (a mention isn't a fact-judgement). "Only seed drivers seen >2 times" → dropped. This section is the authority: if any older wording conflicts, this wins.

The Driver: a tiny, permanent label.

The class node stays **small forever** — just enough to name the cause and route everything else. Five fields, two kinds of link.

id	code-built stable key, e.g. <code>driver:same_store_sales</code> (code builds it, the AI never does).
name	the lower_snake cause, e.g. <code>same_store_sales</code> . A <code>SAME_AS</code> head <i>is</i> the canonical name.
created	when the driver first appeared.
definition	a plain one-line meaning. optional
fact_type	the driver's permanent KIND — one of 4 (next slide). Routes everything downstream.

`SAME_AS` is a *link*, not an alias list

When two names mean the exact same cause, they stay as **two separate nodes** joined by a reversible `SAME_AS` link (a "star": each variant points to one head; the head itself has **no** link out — it is its own canonical; no chains; the head never moves; nothing is ever deleted).

```
customer_transactions —SAME_AS→ guest_count (the head)
comp_sales           —SAME_AS→ same_store_sales
```

Why not a flat aliases array? Because each variant keeps its *own* evidence and its own facts. An array would flatten that away. Trading code just follows `SAME_AS` to the head and unions the whole star (every variant's evidence + DriverUpdates).

Optional links — the driver's "wiring to the books" **best-effort**

Three optional **links** on the class (never per-fact): `MAPS_TO_CONCEPT` → an XBRL accounting concept · `MAPS_TO_MEMBER` → a segment/brand · `MAPS_TO_GUIDANCE` → a company-guidance metric. They're attached only on a confident match, **never block** the catalog, and self-heal on re-run.

▼ **Fine print: what's deliberately NOT on the class**

- **No `evhash16` (value-hash) on the class** — a label is never re-extracted, so there's nothing to change-detect. (The fact and verdict carry one; the label doesn't.)
- **`MAPS_TO_GUIDANCE` is one-to-many** — one driver can union several fragmented guidance synonyms (e.g. `revenue_guidance` → both `guidance:revenue` and `guidance:net_sales`). More on this in slide 39.
- **Links are MATCH-only** (a bad guess makes no link, never a new node) and use *set-replace* on re-run (stale links are deleted, not left behind).
- **The state, numbers, quote, and scope live on the `DriverUpdate`** (the instance), never on the class. ("scope" = which version of the fact — a period/segment; defined on slide 36.)

Every driver has exactly ONE permanent kind.

`fact_type` is the driver's KIND, set once at build time. It does one powerful job: it **routes** which "what happened" words are even legal for that driver (slide 32).

fact_type	the driver is about...	examples
<code>metric</code>	any standing variable you can re-read over time — a number, cost, rate, ratio, count, <i>or</i> a qualitative condition (weather, sentiment, policy in force)	<code>same_store_sales</code> · <code>commodity_cost</code> · <code>consumer_sentiment</code> · <code>tariff_policy</code>
<code>guidance</code>	the company's own forward outlook / target / forecast	<code>eps_guidance</code> · <code>revenue_guidance</code>
<code>surprise</code>	an actual result vs an expectation (consensus or the company's own prior guide) — <i>not</i> vs last period	<code>eps_surprise</code> · <code>revenue_surprise</code>
<code>action_event</code>	a discrete thing that happened — a decision, deal, incident, approval, one-off charge	<code>dividend</code> · <code>restaurant_closure</code> · <code>ceo_succession</code>

The one test that splits `metric` from `action_event`

Between two events, is there a **standing level you could re-read** ("what is it now vs before?")?

YES → `metric` ("what is the cost now?") e.g. `commodity_cost`
 NO → `action_event` ("did it happen / get undone?") e.g. `asset_impairment`

A `_surprise` or `_guidance` framing overrides this to those kinds.

Dual framing is allowed

The same topic can be two drivers of different kind, decided by the *name*: `dividend` (`action_event`) vs `dividend_per_share` (`metric`).

Who sets it & when

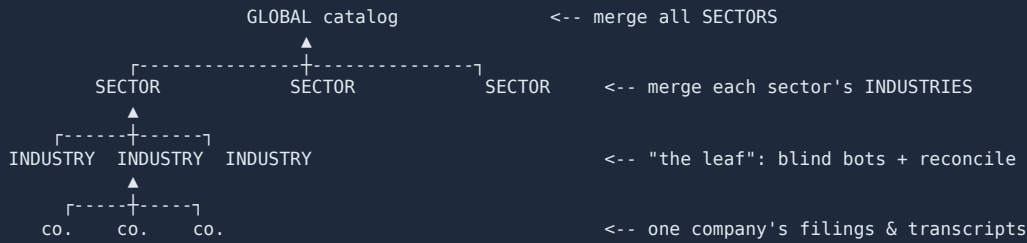
A **strong model (Opus)**, as a final build step, **once per driver** (never per event). Cheap, because it runs once. The blind reader never sets it.

▼ Why a routed kind, and why we won't add more rules

- **Why not one flat up/down list?** ~1 in 6 real facts have no up/down (risk talk, plain snapshots, "stabilized", guidance ranges). Routing by kind handles them cleanly.
- **Why distinct kinds matter for trading:** each is its own query signal — "show me all `withdrawn` guidance" = bearish — so we deliberately do *not* collapse them.
- **Validated on all 1,282 real driver names** — 0 fit none; every "merge to 3" or "add a 5th" attempt failed for a concrete reason.
- **Names that read both ways → default to the standing bucket:** `litigation` , `convertible_notes` , `dividend_policy` , `restructuring_costs` → `metric` ; `corporate_restructuring` , `asset_impairment` → `action_event` .
- **Keep the 4 definitions exactly as written — add no extra clauses or examples.** A tested extra clause overfit (fixed two cases, broke one). Opus was stable on 28/28 keyed cases; a cheap model wobbled.
- **It's a precondition:** a driver with no `fact_type` yet → any DriverUpdate write is rejected.

Build from the bottom up, like merging folders.

Start at one company. Merge companies into an **industry**. Merge industries into a **sector**. Merge sectors into the **global** list. The **same** merge engine runs at every level.



New names: only at the bottom

The **only** place names are invented is the leaf (one company at a time). Every level above just *consolidates* — it never invents new drivers (one rare exception: splitting a true homonym).

Keep each step small

Each box must be small enough for an AI to judge carefully. If a box is too big, code **splits it into batches**. Levels are just batches of batches.

Real shape of the full tree: 11 sectors → 115 industries → ~796 companies.

Everything flows as one simple shape: a "record."

One record = one driver, plus the real quotes that prove it. This is the only shape in the whole system.

```
{
  "driver_name":      "same_store_sales",      // the cause, as a plain lowercase noun
  "canonical_name":   "same_store_sales",      // itself, OR the name it merged into
  "companies":        ["CAKE"],                // who has this driver
  "evidence_refs": [    // WHY it exists – real quotes
    { "company": "CAKE", "source_type": "10-K",
      "source_id": "0001104659-26-018643", "date": "2026-02-23",
      "quote": "...comparable restaurant sales trends." }
  ],
  "same_as_variants": [],                      // other names that mean the same thing
  "optional_links":   { "xbrl_concept": null, "xbrl_member": null, "guidance_ref": null } // the 3 links from slide
8, flattened into the record
}
```

The atom: 5 fields that ID one quote

Each piece of evidence is identified by five fields together: `company` + `source_type` + `source_id` + `date` + `quote`. Code dedups, sorts, splits, and verifies **everything** by these five fields. If you remember one thing, remember this is the unit of truth.

This is the **build artifact** — the catalog file. In the live graph it becomes the **Driver node** (slide 8): `same_as_variants` turn into `SAME_AS` links, and `canonical_name` just marks the link's head. Notice what's *not* in the name — no direction, number, or date; those live on the **DriverUpdate** (slide 31).

Two kinds of files — and the split is the design.

Don't memorize file names. Just know there are **AI orchestrators** (ask the AI for meaning) and **code tools** (do the bookkeeping and refuse to ship anything broken).



AI orchestrators

`.js`

- `menu_build.js` — runs the leaf
- `reconcile.js` — the merge engine
- `fold_catalogs.js` — merges levels up
- `build_tree.js` — the master conductor
- `repair_duplicates.js` — catches missed merges
- `gate.js` / `catalog_first.js` — for live use later



Code tools

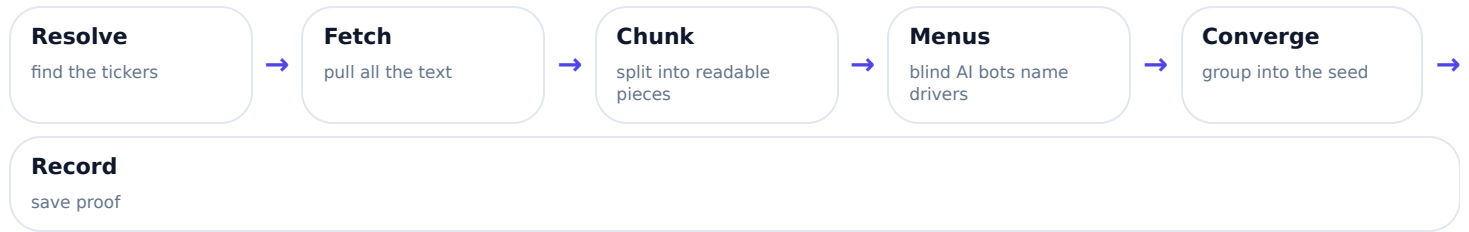
`.py`

- `resolve_driver_scope.py` — list tickers
- `fetch_company_sources.py` — pull text
- `chunk_company_sources.py` — split text
- `build_seed.py` — group into the seed
- `slice_seed.py` — batch a big seed
- `assemble_catalog.py` — **write** the catalog
- `validate_catalog.py` — the hard gate
- `fold_catalogs.py` — deterministic merge math

Rule of thumb: a `.js` file *asks the AI*; a `.py` file *does the math and checks it*. The AI files have no filesystem of their own — they hand small lists to the code files.

Turn one industry into a raw catalog, in 6 steps.

Run `menu_build.js` with `{ industry: "Restaurants" }`. It does this, in order:



The next 5 slides walk these one at a time. Output of the whole stage = `seed.json` : a raw, still-messy first draft of the catalog. Cleaning it up is Stage 2.

▼ What "save proof" (Record) means

The last step writes a `manifest.json` — the exact git code version, the ticker list, the seed's fingerprint, and counts. So months later you can prove exactly what code and data produced this catalog. Everything is reproducible.

"Which companies are in this industry?"

Ask the database (Neo4j) for the ticker list.

resolve_driver_scope.py

```
--industry "Restaurants" → ["BLMN","CAKE","CMG","DRI","EAT","MCD","SBUX","TXRH","WING","YUM", ...]  
                          (14 restaurant tickers)
```

Guards

0 tickers found → **hard stop** (you typo'd the name).

Only 1 ticker → warn (can't measure agreement with one).

Code-to-code

The ticker list is written to a file the next tool reads directly — so an AI relay can never silently drop a company from the run.

Pull every company's real words — and clip nothing.

For each ticker, grab all the non-news sources **with full text**.

`fetch_company_sources.py`

What gets pulled

Earnings call transcripts (prepared remarks + every Q&A) · 8-K press releases · 10-K / 10-Q business, risk, and management-discussion sections · the company's reported KPI names.

The big change: stop dropping text

The old version had size caps that quietly clipped **~62% of the text** (and **100% of transcripts**). That's a huge recall

loss. Now it pulls everything. One real company (CAKE) = **2.24M** characters of source text.

"News" is deliberately *not* in this seed — news drivers get added live later (slide 38). The seed is built only from what the company itself reports.

An AI can't read 2 million characters. So split — never clip.

Cut the text into **~40,000-character pieces**, always at a clean seam.

`chunk_company_sources.py`

The fallback ladder (it always finds a cut point)

- (a) pack whole small events together
- (b) too big? → split at natural seams (a section, or one Q&A exchange)
- (c) still big? → split by paragraph
- (d) still big? → split by sentence
- (e) still big? → split by raw character count ← always works

The promise: not one byte lost or changed

Glue the pieces back together and you get the **original text, exactly** — checked by a hash. The chunker only *slices*; it never rewrites. So splitting big filings costs us nothing.

One blind AI bot per piece — the only place names are born.

Each bot sees **only its one chunk** — no other company, no shared list. It reads the real text and coins specific driver names.

Real example (a CAKE bot)

```
raw KPI label: "North Italia Comparable Restaurant Sales"
               | bot applies the naming rules
               v
driver_name:   north_italia_same_store_sales
evidence_quote: "North Italia Comparable Restaurant Sales"
source_type:   fiscal.ai-kpi
```

Naming rules (plain)

- Name the **cause only**, as a noun:
oil_price , restaurant_traffic .
- NO ticker, date, number, or up/down word in the name.
- Vague text → skip (never invent).

Why "blind"?

If a bot saw the shared list first, it would lazily reuse a near-match. Blind naming + careful merging later is what makes two independent bots actually *agree* — instead of being nudged into fake agreement.

▼ The full naming rulebook (the fine print)

- **Banned inside a name:** state words (beat, cut, rose), direction (long/short), dates/quarters, numbers/units (bps, %, usd), any company ticker or person's name, source labels, accounting prefixes, metaphors, and bare category words (demand , macro , sentiment alone).
- **Earnings convention:** name the metric + its mechanism — eps_surprise (reported vs expected) or revenue_guidance (forward outlook). "beat / raised" is a *state*, not part of the name.
- **Brand ≠ company-wide:** taco_bell_same_store_sales is its OWN driver, separate from same_store_sales . They are never merged.
- **Reusable class, even if seen once:** government_shutdown is fine. A name glued to one single event (q1_2026_shutdown_effect) is rejected.
- **Two causes in one quote?** Emit two driver names, never one bundle.

▼ **Anti-cheat: the bot writes its menu to a file too**

Each bot both *returns* its menu and *writes* it to disk. A later count cross-check fails the run if the two disagree — so a bot can't quietly return something different from what it saved.

Code groups all the menus into one raw seed.

No AI here. Just bookkeeping: group identical names, merge their evidence.

`build_seed.py`**2,246**

raw candidate names from all bots

**group + dedup**

by lowercased name & by the 5 fields

**594**

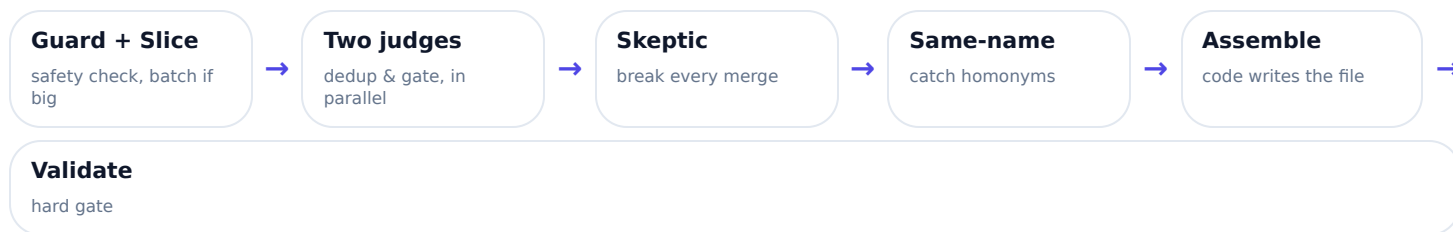
distinct drivers in the seed

That `seed.json` — 594 driver records, each backed by real quotes — is the raw draft. It still has duplicates and junk. **Stage 2 cleans it.**

"Group by lowercased name" is the only thing that merges here — exact spelling matches only. Anything subtler (synonyms) is left for the careful judges in Stage 2.

The merge engine: clean the seed into a real catalog.

Run `reconcile.js` . This is the heart of the system — and it runs at **every level** later, too.



Next 6 slides walk these. Output = `catalog.json` : the clean, proven list.

Don't get billed wrong, and don't overload the AI.

Billing guard (step 0 of every workflow)

Check: is an API key sitting in the environment? If yes → **refuse to run**. Everything must go through the subscription, never the metered pay-per-call API. (A hard rule from your setup.)

Slice into batches

`slice_seed.py`

594 records is over the safe limit (400 records / 300k chars). So code cuts it into **name-sorted batches** so each AI judge reasons over a small, careful pile.

Trade-off: two synonyms with no shared words might land in different batches and miss each other. That's an *under-merge* (the safe direction) — and the repair pass (slide 29) catches it later.

Two independent AI agents, working in parallel.

```
seed batch -- DEDUP → "which names mean EXACTLY the same? → propose merge links"
           -- GATE  → "is each name a valid, reusable driver? → admit / rewrite / skip"
```

Dedup — the matcher

Proposes "these two are the same" links and picks one survivor name. Different brand, scope, or metric → rejected, kept separate.

Gate — the bouncer

An *independent* judge (doesn't trust the bot that coined it):

- **admit** — valid driver
- **rewrite** — fix wording only
- **skip** — vague or one-off

Both judge from the **evidence quotes**, not the bare name string.

▼ The "3-check" every merge must pass

To call two names the same, all three must be true: **same object** (the thing) · **same scope** (brand vs company-wide, geography) · **same mechanism** (how it acts). If any one is false or unclear → keep separate. A "rewrite" may only fix wording — never change the meaning.

A third agent whose only job is to break the merges.

This is what stops the v2 collapse. For every proposed merge, the default answer is **NO**. It flips to YES only if the skeptic *cannot* refute that they're the exact same **object + scope + mechanism**.

Then code — not AI — filters

Refuted links are dropped. Refuted rewrites are parked. A **missing** answer counts as refused. Fail-closed, always.

High-blast: a second skeptic for big merges

A wrong merge across 2 companies is small; across **hundreds** it's a disaster (exactly how v2 died). So any surviving merge touching **≥ 8 companies** gets a **second, independent** skeptic that must separately pass object · scope · mechanism, each backed by a quote. Either one says no → the merge dies.

▼ Why the company count is computed by code

The "how many companies?" number is computed by a tiny script and only *relayed* by the AI, with a checksum. A deflated count could skip the second skeptic — so the validator later **recomputes it from scratch** and fails the run if a **≥8-company** merge is missing its second-skeptic proof.

One name, two meanings? Catch the rare "homonym."

If a single name's **own** evidence shows two different real-world meanings across companies, it goes to a tiny review. Usually this fires **zero** times — but it's the safety hatch.

SAME

False alarm — one real cause. Keep it (skeptically-confirmed).

DIFFERENT

True homonym. Split into more-specific names, drawn only from the evidence.

UNCLEAR

Too thin to tell. **Park** it — leave it out of the clean list (fail-closed).

You'll see all three of these verdicts on real data in slide 27 — they're the same verdicts the higher merge levels use.

Code writes the catalog — closing the biggest hole.

The AI agents only produced **short decision lists**. Now pure code applies a fixed 5-way rule to each seed record.

`assemble_catalog.py`

```
For each driver in the seed, code asks in order:
1. gate said skip?      → put on the skips[] list
2. approved merge?     → canonical_name = the survivor
3. approved rewrite?   → canonical_name = the fixed name
4. parked rewrite?     → unresolved[] list
5. otherwise           → admit as-is
```

Why this is the key safety move

Because **code** writes the file from an approved list, the AI **cannot invent a merge** the skeptic never approved. The v2 mush is now structurally impossible — not just discouraged.

Real result: 594 seed names → a 573-record catalog

The 573 records = **388 canonical survivors** + **185 variants** linked back by `SAME_AS` . Plus **7 skipped** and **16 parked**, kept on side-lists (out of the clean catalog).

The run's raw decision tallies — **keep 388** **same_as 175** **rewrite 10** **skip 7** **parked 16** — **overlap**

(a rewrite can also be a `SAME_AS`, and the same-name check on slide 23 splits a few homonyms), so they don't add up to a flat 594. The validator (slide 25) still proves every seed name is accounted for exactly once.

```
acquisition_related_costs  |
acquisition_related_expenses |---> acquisition_expenses  (3 names → 1, read in one hop)
frc_acquisition_costs      |
```


The hard gate: re-check structure, hard-stop on any flaw.

Zero judgment — just structure. If anything is off, the run **fails** (the catalog can never ship broken).

`validate_catalog.py`

What it checks

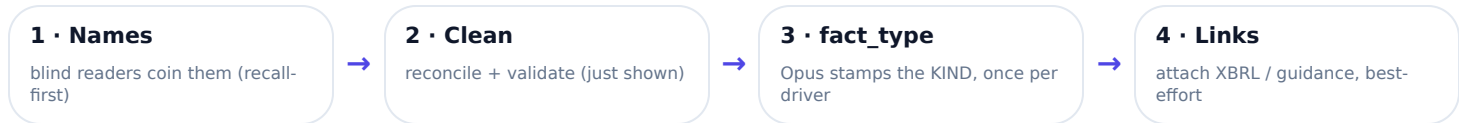
- **Every merge is approved** (the "D1" check): a merged name must trace back through the approved-link list. Code can't justify it → fail.
- **Nothing lost or duplicated** — every seed name is accounted for exactly once.
- **No invented names; evidence matches the seed exactly** (down to the quote).
- **Re-computes the ≥8-company list itself** and demands the second-skeptic proof.

The receipt

It writes `validation_exit.json` — a signed note saying "passed, and here is the catalog's fingerprint." Later steps refuse to use a catalog unless this receipt says **passed** and the file hasn't changed since. This catches "the check never ran" and "someone edited it afterward."

Two last steps make each driver "born complete."

The names are now clean. Before a driver counts as finished, the build adds **its kind** and **its wiring** — in the *same* run. Still **no facts** are created here.



Step 3 • fact_type (mandatory)

A **strong model (Opus)** reads each finalized name + its evidence and assigns one of the 4 kinds (slide 9). **Once per driver, not per event** → cost is trivial. A separate classifier; no re-reading the source.

Step 4 • optional links (best-effort)

Attach `MAPS_TO_CONCEPT` / `MAPS_TO_MEMBER` / `MAPS_TO_GUIDANCE` only on a confident match. A miss **never blocks** the catalog and **self-heals** on the next run.

Why one run, not a later pass

Because the **same** recipe runs when a *live producer* coins a brand-new driver on the fly — and a live producer has no "later pass." So a driver is always **born complete**: name + `fact_type` + links, whether it's the first big build, an incremental refresh, or one new name mid-stream.

Still zero DriverUpdates

This finishes the **label**. The **facts** (DriverUpdates) and **verdicts** are made later, only by a producer reading a real event — that's the producer's job (slides 31-37).

▼ Why Opus for the kind, and why the rules stay frozen

Opus was stable and correct on **28/28** keyed cases plus all the "reads-both-ways" cases; a cheap model wobbled (locked 2026-06-16). Keep the 4 definitions **exactly as written** — a tested extra clause overfit (fixed two cases, broke one). In an incremental refresh, a driver's `fact_type` is **frozen like its name** — old drivers keep theirs; only brand-new drivers get a fresh one.

Merging upward — shown on real data.

To build a sector, merge its industry catalogs. Same engine, bigger units. Here's an actual test fold of 4 child catalogs — watch **one word get three different fates**.

The children (4 industry catalogs)

```
child_a: alpha_unique, comparable_sales, delivery_mix, oil_price, same_store_sales, traffic
child_b: beta_unique, delivery_mix, oil_price, traffic
child_c: foot_traffic, gamma_unique, oil_price, wage_inflation
child_d: delta_unique, fulfillment_cost, wage_inflation
```

oil_price → SAME

Both quote "crude oil price." The different downstream costs (truck fuel vs jet fuel) are company-specific, not different drivers. → **one merged record**.

delivery_mix → DIFFERENT

Child A = restaurant order channels. Child B = parcel shipping modes. True homonym → **split** into
 restaurant_delivery_channel_mix +
 parcel_delivery_mix .

traffic → UNCLEAR

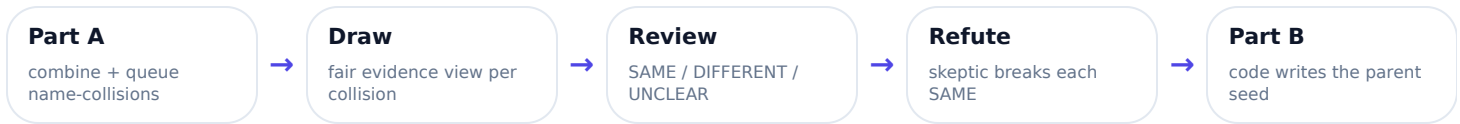
Both quotes are empty ("traffic was a factor").
 Can't tell → **park** it, leave it out of the clean catalog. Fail-closed.

Same spelling, three fates, each decided from evidence and defaulting to "keep separate" when unsure. That's the whole philosophy in one picture.

How the fold actually runs.

A deterministic **combine** step sits in front of the same reconcile engine.

`fold_catalogs.js` / `.py`



Part A: pass-through vs collision

A name in only **one** child passes straight through. A name in **2+** children is *not* merged — it's queued for the review above (identical spelling $\neq$ identical meaning).

Draw: surface the disagreement

Shows the skeptic a fair sample of quotes — **smallest side first**, rarest companies first. At scale, a bad merge usually slips through only because the one contradicting quote was never shown. This forces it into view.

▼ What carries forward, and the bookkeeping that keeps it honest

- Merge links from lower levels are **kept** and never reopened; new cross-child merges are added through the same skeptic. **Why that's not a contradiction:** the skeptic is a one-time gate at a merge's *birth*, not a recall — a merge is judged once when first proposed, then carries forward un-re-judged. Each level only judges the brand-new pairings that two catalogs sitting side by side make newly possible.
- A merged name's old record stays recoverable in the committed child file — merges are **reversible**, never a destructive delete.
- The fold writes a `fold_manifest.json` recording, per child, how many records were kept vs dropped — so nothing vanishes silently.
- The fold's `validate` adds checks (called "D8") that parent evidence = the **exact union** of child evidence, and that a split perfectly partitions the original quotes.

One conductor, plus two cost-savers.

build_tree.js — the conductor, with 3 modes

`{ list: true }`

Read-only: print the Sector → Industry → Ticker tree; fail loud if any company has two parents. Never merges.

`{ fold: {...} }`

Merge specific children once → reconcile → fold-check → repair → fold-check.

`{ walk: {...} }`

The full climb: fold every sector, then global. Every step fail-closed — one failure kills the whole run.

Repair pass

`repair_duplicates`

Catches merges the batching missed. Code *suggests* look-alike pairs (shared rare words, or cheap embeddings — suggest only); AI judges; code applies. It only **adds** links, never breaks one.

Resume

`resume_menus`

A real run = ~950 AI bots and can hit the 5-hour subscription window. If it dies, relaunch with `resume_run_id` → it re-runs **only the missing bots**. Finished work is never paid for twice.

"Embeddings" = a cheap math trick that scores how similar two names *feel*. Here it only **suggests** pairs to look at — it never decides a merge. The AI + skeptic always decide meaning.

Pick the right AI for each job — recall vs cost.

Different AI models behave very differently. Picking the wrong one quietly loses drivers *or* wastes money. So each job gets a specific model.

Measured fact: reading is model-dependent

On the same chunk of text, the models found wildly different numbers of candidate drivers:

```
Fable 5 → 54 candidates ◀ best recall (we use this for the readers)
Sonnet  → 31 candidates
Opus    → 24 candidates
Haiku   → noisy + breaks naming rules
```

A driver missed at the reading stage is **gone forever** — nothing downstream can recover it.



Reader bots → Fable

Their job is to *not miss anything*. Recall is everything, so they use the highest-recall model.



All judges → Opus

Dedup, gate, skeptic, folds, validators — the careful decisions use Opus: the model the design was validated on, and cheaper than the high-recall reader.

Plus: the billing guard (subscription-only) runs on every workflow, and big bot bursts get a fresh time window so they don't run dry — with resume as the safety net if one dies.

The DriverUpdate: one real fact, written by a producer.

Everything so far built the **labels**. Now the **producer** reads a real event and, when it finds a genuine fact, writes one DriverUpdate — the instance that hangs off a label.

```
{
  "id": "du:CAKE_2026-02-23:same_store_sales:period=fy", // code-built key
  "driver_state": "increased", // what happened (from the driver's lane - slide 32)
  "level_low": 3, "level_unit": "percent_yoy", // the size (slide 33)
  "comparison_low": 1, "comparison_baseline": "prior_year",
  "quote": "comparable sales increased 3% vs 1% a year ago",
  "source_type": "8-K", "date": "2026-02-23", // where + when it was said
  "fact_scope": "period=fy", // which version of the fact (slide 36)
  "evhash16": "a1b2c3d4e5f60718" // value-fingerprint (slide 36)
}
// links: -OF_DRIVER→ (Driver same_store_sales) · -FROM_SOURCE→ (Event 8-K)
```

The GATE: a mention is not a fact

Before writing anything, the producer asks: *does this event carry a real state / change / surprise / guidance / action about the driver?* If it's just a passing mention → **write nothing**. ("Weather may affect results" is boilerplate → dropped.)

Who fills it

The catalog reader writes **none** of this (it only saved a name + a quote). The **producer** fills `driver_state` + the numbers, because only it has prior-period memory and the real event.

A single event is enough

One event can be a whole fact (a CEO exit, an FDA nod). Every fact is kept — even a driver's very first, even a one-off. "Seen often" is a read-time view, never a write rule.

▼ The smaller nuances on the fact node

- **quote** is stored only when the source gives a real state/value/change — and the verbatim quote is always the **final truth**; the state + numbers are just queryable helpers.
- **Source & timing:** `source_type` (where it came from — e.g. "8-K"), `date` (when it was said), `created` (when we wrote it). (only the last two are timestamps.)
- **Two links are always present:** exactly one `OF_DRIVER` (to its label) and one `FROM_SOURCE` (to its event).
- **The same fact reported in two events** = two DriverUpdates (one per event); they're only merged in a read-time view.

"What happened?" — pick ONE word from the driver's lane.

`driver_state` isn't free text. The driver's `fact_type` opens **one lane**, and the producer picks a single word from it. Off-lane words are rejected by code.

fact_type lane	allowed driver_state words
<code>metric</code>	<code>increased</code> · <code>decreased</code> · <code>unchanged</code> · <code>mixed</code> · <code>reported</code> · <code>persists</code> · <code>unknown</code>
<code>guidance</code>	<code>introduced</code> · <code>raised</code> · <code>lowered</code> · <code>reaffirmed</code> · <code>withdrawn</code> · <code>unknown</code>
<code>surprise</code>	<code>beat</code> · <code>in_line</code> · <code>missed</code> · <code>unknown</code>
<code>action_event</code>	<code>at_risk</code> · <code>announced</code> · <code>occurred</code> · <code>continued</code> · <code>resolved</code> · <code>canceled</code> · <code>suspended</code> · <code>unknown</code>

The `metric` ladder — pick the FIRST that matches

- | | |
|--|--|
| 1. direction stated (rose / fell, more / less) | → increased / decreased |
| 2. same driver up in parts AND down in others | → mixed |
| 3. explicit flat / steady / same-as-before | → unchanged |
| 4. ongoing condition, no up or down ("still elevated") | → persists |
| 5. a bare value, no comparison | → reported (a prior value present → increased/decreased) |
| 6. a real fact but no readable state | → unknown |

Ignore good/bad — that's the verdict, not the state. "Adverse weather worsened" → `increased` (more of it), even though it's bad.

`at_risk` — strict, or it floods

Only a **specific, current, source-flagged** adverse event that hasn't happened and isn't the company's own plan (`debt_covenant_default` , `delisting` , `loss_of_license`). A planned closure is `announced` , not `at_risk` . Generic risk boilerplate → the gate drops it.

Outlook verbs route to guidance

"expect / anticipate / target / plan to ___" → a `guidance` state (raised / lowered / reaffirmed), never a metric state. `surprise` is measured vs an *expectation* (consensus or own prior guide), never vs last period.

▼ Two word-choices that prevent collisions, + how it's enforced

- **mixed** is ONE driver moving both ways. Two *different* drivers moving opposite ways are not "mixed" — they're split into two drivers, each with its own state.
- **narrowed** **is not a state** — it's derived from the comparison numbers at read time. And the metric word is **persists**, not **continued** (which would steal "continued to grow" from **increased**).
- **Lane check (code, hard-fail, zero AI):** at write time code asserts the state is in the driver's lane (and the units + **comparison_baseline** are on-menu). The producer *proposes*; code *decides*. Off-lane → rejected, no write. (Done in app code because Neo4j can't check one field against another.)
- **Locked & measured:** the 4 lanes were validated on **3,825** real evidence quotes.

The size of the move — in 9 fill-only-what-applies slots.

A word like "increased" loses the size. So the producer also fills numbers — but **every slot is optional**. All empty = a qualitative fact where the quote is the truth. **Never invent a number.**

One filled example first — so the slots aren't abstract

```
"Margin rose 60 bps to 17.6%"
→ level_low = 17.6  level_unit = percent      (the resulting level)
→ change_value = +60 change_unit = basis_points (the move itself)
```

the level (3 + unit)

`level_low` · `level_high` · `level_bound` (`floor` / `ceiling`) · `level_unit` — the resulting value, or a range, or an open bound.

the move (2)

`change_value` (the **signed delta** itself, e.g. `+60`) · `change_unit` (may differ from the level's unit).

the baseline (3)

`comparison_low` · `comparison_high` · `comparison_baseline` — one of `consensus` / `prior_year` / `sequential_period` / `previous_guidance` (OR `null`). `comparison_high` is set only if the prior is itself a range; `null` = a single-point baseline.

The unit menu (one shared list)

m_usd	\$ millions (6135 = \$6,135M)
usd	\$ absolute (1.20)
percent	a % level (17.6% margin)
percent_yoy	a year-over-year % change
percent_points	a move in % points (+0.6 pts)
basis_points	a move in bps (+60)
count	a number of things (stores)
x	a ratio / multiple (2.5x)
unknown	unit not determinable

The "shape" is read from the slots

```
point    level_low set, rest null    AUV = $12.2M
range    level_low + level_high    $90M ... $100M
floor    level_low + bound=floor    "at least $5.00"
ceiling  level_high + bound=ceiling "up to 2.0x"
none     all four null             "dividend suspended"
```

No extra "shape" field — the filled slots tell you which it is.

Why this exact unit list?

It's copied **verbatim** from the live Guidance system's units — so a driver's numbers line up with its guidance link (slide 39) with **no translation table**.

▼ Fields we deliberately did NOT add (to stay minimal)

- **No** `mid` — derive it from low/high.
- **No** `usd_per_share` — "per share" lives in the driver name (`eps`), so the unit stays `usd` .
- **No** `unit_raw` — the raw wording is already in the `quote` .
- **No GAAP-vs-adjusted field** — that lives in the name (`adjusted_eps` $\neq$ `eps`).
- **No free-text** `qualitative` and **no** `change_high` — the quote carries number-less or ranged-move facts.

Seven rules keep the numbers honest — four examples show them.

Code enforces these at write time: the producer **proposes** the numbers, code **decides** if they're legal and rejects anything off.

The 7 hard rules (plain)

1. **Open bounds null the other end.** `floor` → no `level_high` ; `ceiling` → no `level_low` .
2. **A stated delta's sign must match the state.** + for increased/raised/beat; – for decreased/lowered/missed. (states with no direction — `reported` , `persists` , `unchanged` — impose no sign rule.)
3. **Store a baseline only if the source says it** (a frozen fact of this event) — never compute it from another node. Set it `null` for "vs peers", a fixed anchor year, or a streak ("third straight quarter"); if a stated baseline fits no menu value, keep the number but set the baseline `null` .
4. **Per-share / per-week lives in the NAME** (`eps` , `sales_per_square_foot`); the unit stays the base (`usd` / `count`).
5. **One primary baseline only.** If a fact cites several, keep the headline one (tiebreak: `prior_year` > `sequential_period`); the rest stay in the quote. "Exceeded expectations" reads by kind — a `surprise` → `consensus` ; a metric/guidance "beat our guide" → `previous_guidance` .
6. **Store `change_value` only if the source states the move.** If it's derivable from level + baseline, leave it null (no drifting third copy); a *ranged* move stays in the quote (there is no `change_high`).
7. **Rate-vs-level routing.** A % that *is* the metric goes in the **level**; a % move on a \$-metric goes in **change**.

Four worked examples

```
A "Margin rose 60 bps to 17.6% from 17.0% a year ago"
state=increased · level_low=17.6 percent · change_value=+60 basis_points
comparison_low=17.0 · comparison_baseline=prior_year

B "Raised FY guidance to $90-100M from $85-95M"
state=raised · level_low=90 level_high=100 m_usd
comparison_low=85 comparison_high=95 · comparison_baseline=previous_guidance

C "Dividend suspended"
state=suspended · ALL number + comparison slots null · the quote is the fact

D "EPS of $1.30 beat the $1.20 consensus"
state=beat · level_low=1.30 usd · change_value=+0.10 usd
comparison_low=1.20 · comparison_baseline=consensus
```

A `surprise` 's `change_value` = actual – expectation (example D: 1.30 – 1.20 = +0.10).

Rule 7 in action: `same_store_sales` "rose 3%" → `level_low=3`, `percent_yoy` (the 3% *is* the metric). But `revenue` "rose 3% to \$6.1B" → `level_low=6100 m_usd` , `change_value=+3 percent_yoy` (the metric is dollars; the % is the move).

The verdict: did this fact actually move the stock?

A fact can exist with no verdict. But when the producer judges that a fact **drove** the move, it clips a verdict onto the `Event → DriverUpdate` link (`EXPLAINED_BY`).

```
(Event 8-K) -[:EXPLAINED_BY]→ (DriverUpdate: same_store_sales increased 3%)
{
  stock_impact: "long",           // direction it pushed (can oppose the net move)
  weightage:    0.4,              // standalone force 0.1..1.0 – or null to abstain
  confidence:   80,               // how sure it's TRUE, 0..100
  produced_mode: "live",         // judged in real time – point-in-time, no future info
  llm_producer: "earnings-learner"
}
```

Three independent dials — none derivable from another

- stock_impact = direction** `long` or `short` . Which way this fact pushed — it can push opposite to where the stock actually went.
- weightage = force** how strong this one driver was, `0.1..1.0` . A **standalone** estimate — **not** a share of the move.
- confidence = certainty** how sure the producer is the attribution is **true**, `0..100` . Separate from force.

Why **weightage** is independent (this is the subtle part)

Forces are **never summed to 1** and adding a new driver **never rewrites** another's weightage. A move can be **partly unexplained** (market / sector / noise) — so a sole driver is **never** auto-set to max. Unsure of the size? Set `weightage = null` (abstain) — keep the direction.

Shares are computed only when you read: `share = weightage ÷ (sum of weightages on that event)` ; `signed_force = weightage × direction` . Neither is ever stored.

Point-in-time, and graded fairly

The **real** share / actual return is **never stored** on the verdict and **never shown to the predictor**. Reality gives one net return, so a verdict *set* is graded on net sign/size + ranking — never a per-driver "true share." A `live` verdict (judged before the move) always beats a `backfill` one at the same key.

▼ The verdict's identity & validator

- **Key = event + driver + fact_scope + producer.** (`fact_scope` = which fact within the event; slide 36.) The *producer is in the key*, so two producers can disagree without colliding. No company (it's reached via the event).
- `evhash16` = a hash of exactly (`stock_impact` , `weightage` , `confidence`). `produced_mode` is excluded — a live↔backfill swap isn't a judgement change. So the hash flips only on a real change to *this* edge.
- **Validator (hard-fail):** `stock_impact` ∈ {long, short} ; `confidence` ∈ {0,10,...100} ; `weightage` ∈ {0.1,...1.0} or `null` . Both sit on a 10-step ladder — no false-precision decimals, so the hash stays stable across re-runs.
- **The name:** we call it the *attribution* (chosen over "impact"); there is no separate verdict box — it's purely the edge.

Code builds every key. Re-runs overwrite — never duplicate.

How does the graph avoid piling up duplicates when a producer runs twice? Every node gets a **code-built key** with a uniqueness rule, so a re-run **updates in place** (a "MERGE"), never forks.

Fact key

`event + driver + fact_scope` — **no producer**, so two readers of the same fact land on **one** node.

Verdict key

`event + driver + fact_scope + producer` — producer added, so two producers can disagree without colliding.

`fact_scope` = "which version of this fact, inside this event"

One event can state the same driver twice. `fact_scope` keeps them apart in the key — but it **never** enters the driver name.

```
May 10-Q, driver = same_store_sales, ONE event, TWO facts:
"Q1 comp sales +3%"      → fact_scope = "period=Q1"
"April month-to-date -1%" → fact_scope = "period=april_mtd"
→ two DriverUpdates, not one lost. Still ONE driver: same_store_sales.
```

How it's chosen: (1) a structured scope if the source gives one — period / segment / geography; (2) a hash of the normalized quote only when there's no clean scope. Structured first, because it correctly **merges** a restatement of the same fact (press release + MD&A both "Q1 +3%" → one fact); a raw quote-hash would wrongly split them.

The value-fingerprint (`evhash16`)

A small hash of the node's **values** — a property, **never part of the key**. It flags when something **actually changed** versus a no-op re-run. The fact hashes its payload; the verdict hashes its three dials. (The label has none — it's never re-extracted.)

This whole recipe — class node + per-event instances, code-built keys, merge-in-place, value-hash — is exactly how the live **Guidance** graph already works (548 + 8,432 nodes). We're copying a proven pattern.

The few quiet facts the whole design leans on.

These are the load-bearing assumptions — each one verified, each one the reason a shortcut elsewhere is safe.

Invariants (verified)

- **One company per event** — checked live: News 348,669 · Reports 42,243 · Transcripts 9,608, all exactly 1. So an event names its (company, move); the company never sits on the driver layer.
- **One producer per event** — earnings → earnings-learner , news → news-driver . (If ever broken → parallel verdict links, one per producer.)
- **The real stock move stays on the existing Event → Company link** — never on a driver node.
- **XBRL / guidance links sit on the class**, not per fact.

Persistence

- **Every DriverUpdate is kept** — including a driver's very first, and one-offs.
- **"Only show drivers seen N+ times" is a read-time filter**, never a write-time drop. Nothing is ever lost; re-runs stay deterministic.

One line to remember

The driver layer stores **causes and facts**; the **company and its return** are always one hop away through the event. That single choice removes the company from every key and needs no special case for market-wide drivers.

Reminder: none of this graph is built yet — 0 Driver / 0 DriverUpdate nodes exist. What's locked vs still open is covered in slide 40.

Building the catalog is once. Using it is forever.

Once the catalog exists, every new event runs through a **producer** (slides 31–37). First it settles the **name** — reuse an existing driver or coin a new one — then it writes the **fact** (+ verdict). This slide is the name half; it's **still being wired**.

G1 · propose-first

coin your own name BLIND from the evidence



see related names

reuse ONLY on exact match



G2 · gate

a brand-new name still passes the admission gate

Why propose-first?

If you saw the catalog first, you'd lazily grab a near-match. So you name it blind, *then* look — and only reuse if it's truly the same. Same anti-laziness trick as the blind leaf bots.

The honesty gate — the real test

Freeze the catalog. Feed it **fresh** events, showing only info from on/before that date (no peeking at the future). It must reuse / create / skip. An independent grader scores it against a **pre-written** answer key — **graded once**, no retuning to pass.

What the producer does, end to end

```
name it (G1) → reuse OR coin → if new: stamp fact_type on the fly (slide 26)
→ is there a real fact? (the gate) → write the DriverUpdate (slide 31)
→ did it move the stock? → write the verdict (slide 35)
```

The catalog itself is "producer-agnostic": it carries no "news vs fundamental" tag — that's the producer's concern (it lives on `llm_producer`), not a property of the driver. News drivers are coined live, here — never in the seed.

A possible bonus: wire guidance drivers to the live Guidance graph.

There's already a production graph of company guidance (548 Guidance + 8,432 GuidanceUpdate nodes). A `fact_type:guidance` driver could point at it — getting a ready-made time-series for free.

Read this first

This entire slide is a **suggestion** — nothing here is decided, locked, or built (0 links exist). The locked core (slides 5–37) is **not** touched, and the Guidance pipeline is **not** changed. It's a read-only idea for later.

The idea in one picture

```
(Driver revenue_guidance) ==[:MAPS_TO_GUIDANCE]==> (Guidance guidance:revenue)
└──────────┴────────── also unions ─────────┴──────────> (Guidance guidance:net_sales)
```

One driver can point at **several** guidance synonyms (the guidance graph fragmented into leaky synonyms; our catalog is the clean layer it lacks). The link is **MATCH-only** — a bad guess makes no link, never a new node.

How it would match (no hand-list)

(a) **exact slug** (deterministic), then (b) an **evidence judge** that reasons from each anchor's *unit + sample values*, never its name, with a Refute step — **fail-closed** on any doubt. No curated synonym list (proven unsafe on live data).

Measured, honestly

Offline test: **~98-100% precision** (0 wrong links on 87 hard pairs), **~85% coverage**. There is **no** fully-deterministic 100% shortcut — synonym-matching is part-judgement.

The safety asymmetry (why fail-closed)

A **missed** link is a safe gap (just a short series; fixable later). A **wrong** link is the dangerous, hard-to-undo case. So the resolver always errs toward leaving things unlinked.

▼ The fuller idea (resolver, set-replace, born-linked pointer, open decisions)

- **Why it's even possible:** a Guidance node is itself a thin, cross-company, value-free metric tag (a class) — so Driver↔Guidance-anchor is class↔class, and our catalog supplies the canonicalization guidance never enforced.
- **Why no auto alias / auto XBRL-code match:** both are unsafe on live data (227 alias→wrong-anchor collisions; one XBRL code spans 9–11 anchors incl. GAAP + non-GAAP). The evidence-judge reproduces every true synonym from data, so a list is redundant.
- **Set-replace every run:** delete links no longer matched, then write the current set (additive-only would leave stale links).
- **Optional back-pointer:** a derived `canonical_driver` on the Guidance anchor (class-level, never relabels guidance) so a guidance-only tool can find the driver fast. The edge alone is required + sufficient; the pointer is pure convenience and must be re-derived from the edge each run.
- **Instance-level link** (DriverUpdate ↔ GuidanceUpdate) is producer/backfill work, deferred.
- **4 open owner decisions** remain (keep the judge vs slug-only; one-to-many edge; defer instance link; adopt the back-pointer or stay strictly read-only).

Where each part stands — and what "nothing built" means.

The design has tiers. Some parts are **locked** (validated, don't reopen), some **agreed** (settled, minor tweaks possible), some **open**. Knowing which is which keeps us from building too early.

Tier	What's in it	Means
LOCKED	node/edge structure ("Design C") · Driver fields · the 4 <code>fact_type</code> kinds · the 4 <code>driver_state</code> lanes (2026-06-15)	validated; do not reopen
AGREED	the number/comparison layer · the <code>EXPLAINED_BY</code> verdict layer (weightage / confidence / produced_mode) (2026-06-16)	settled; minor owner tweaks possible
DEFERRED	exact <code>id</code> key strings · the "store-number-vs-link-to-XBRL" rule (store the number for now) · wiring the producer to fill state + numbers	decided later, on purpose
SUGGESTION	the whole guidance bridge (slide 39) · instance-level guidance link	an idea; nothing decided

The big one: nothing is built yet

0 Driver nodes and **0** DriverUpdate nodes exist in the database. Everything in the model + live-graph slides (5–9 and 31–37) is a **design + recipe**, not a live system. (The *catalog-build code* from slides 10–30 is built and tested — but it only makes the labels, not the graph.)

Why "locked" is trustworthy

The kinds were validated on **1,282** real driver names (0 fit none); the state words on **3,825** real quotes; the kind-classifier (Opus) was right on **28/28** keyed cases. And the whole shape mirrors the **live** Guidance graph (548 + 8,432 nodes) — a proven precedent.

Why you can trust it — and where it stands.

A handful of patterns repeat everywhere — they are the spine of both the build *and* the graph.

Pattern	What it means
Three layers, never one	a label (Driver) · a fact (DriverUpdate) · a verdict (an edge). Kept separate on purpose.
Judgment → AI, structure → code	AI never writes the file or builds a key; code never guesses meaning.
Build = labels · producer = facts	the catalog makes names + kinds; only a producer (reading a real event) makes facts + verdicts.
Code builds keys; re-runs merge in place	no duplicates, deterministic re-runs; a value-hash flags real change.
Fail-closed	unsure → keep separate / leave unlinked. Missing answer → no.
Skeptic on every merge	each merge is attacked; ≥8 companies gets a second skeptic.
Point-in-time	never show the AI a name, number, or outcome from the future.
Signed receipts	a catalog is usable only if its validation passed and the bytes are unchanged.

✓ Built & tested

The **label factory** (the build code, slides 10–30): leaf, fold, walk, batching, repair, resume. **257+ tests pass** on synthetic / 1-ticker gates. It makes the *names* only.

⚠ Designed, not built

The **live graph** (slides 5–9 + 31–37): Driver / DriverUpdate / verdict, `fact_type`, states, numbers, identity. **0 nodes exist** — recipe, not running.

🕒 Next

Real Restaurants run → 2nd industry → first real fold → honesty gate. Then **wire the producer** (slides 31–37). Guidance bridge (slide 39) is optional.

That's the whole picture — the data model, the build, and the live use. Want a deeper dive on any one slide — e.g. trace a single CAKE driver from raw text all the way to a graded verdict? Just ask.

Every file, one at a time — super simple.

Same shape every slide: **what goes in** → **what comes out** → **the steps** → **why it matters**. We go bottom-up — the **worker tools** first (each does one concrete job), then the **conductors** that string them together.

Worker tools `.py`

43 `resolve_driver_scope.py`
— list companies

44 `fetch_company_sources.py`
— pull text

45 `chunk_company_sources.py`
— split text

46 `build_seed.py` — group
into the seed

47 `slice_seed.py` — batch a
big seed

48 `assemble_catalog.py` —
write the catalog

49 `validate_catalog.py` —
the hard gate

50 `fold_catalogs.py` — the
merge math

Conductors `.js`

51 `menu_build.js` — runs the
leaf

52 `reconcile.js` — the
merge engine

53 `fold_catalogs.js` —
merges levels up

54 `build_tree.js` — the
master conductor

55 `repair_duplicates.js` —
catch missed merges

56 `gate.js` — the admission
gate (live)

57 `catalog_first.js` —
reuse-first (live)

Remember the rule of thumb: a `.py` tool does one job and checks itself; a `.js` conductor asks the AI and calls the tools.

resolve_driver_scope.py

List the companies. Given an industry name, it asks the database which tickers belong to it.

Takes in `--industry "Restaurants"` (or `--sector` , or `--list` to see valid names)

Gives back a list of tickers, and writes them to a small scope file

Step by step

1. Connect to the company database (read-only — it never changes anything).
2. Ask: "which companies have industry = Restaurants?"
3. Return the distinct tickers, sorted: `BLMN, CAKE, CMG, DRI ...`
4. Write that exact list to a file the next tool reads directly.

Its two guards

0 companies found → stop the run (you typed the name wrong). **Only 1 company** → warn (you can't measure agreement with one). Writing the list to a file means an AI can never quietly drop a company.

fetch_company_sources.py

Pull the text. For each company, download all of its real words from filings and earnings calls.

Takes in the ticker list + a run folder

Gives back `sources/<TICKER>.json` (full text) + a list of fingerprints

Step by step

1. For each company, query the database for its 10-K / 10-Q / 8-K filings, earnings-call transcripts (prepared remarks + every Q&A), and its reported KPI names.
2. Keep the **full** text — the old version's size caps clipped ~62% of it.
3. Tag each item with its date, type, and whether the stock moved hard that day.
4. Save each company's bundle, plus a fingerprint (sha256) to prove it wasn't tampered with.

Why it matters

This is the raw material for the whole catalog. **Read-only, nothing dropped.** A driver missed here can never be recovered, so it errs on the side of grabbing everything.

▼ One small bug it fixes

Q&A exchanges are numbered `"0", "1", ..., "10"` as text. Sorting text puts `"10"` right after `"1"` → scrambled order. This tool sorts them as real numbers, so the conversation reads in the right order.

chunk_company_sources.py

Split the text. One company can be 2 million characters — too big for an AI. So cut it into ~40,000-char pieces, always at a clean seam.

Takes in

`sources/<TICKER>.json`

Gives back

`chunks/<TICKER>__chunk_NNN.json` — one bot's reading load each

Step by step

1. Build each event's full text.
2. Small enough? Keep it whole. Pack several small events into one chunk.
3. Too big? Walk the ladder until it fits: natural seam → paragraph → sentence → raw characters (this last one always works).
4. Run `--verify` : glue the pieces back and confirm they equal the original, **byte for byte**.

Why it matters

It lets a bot read everything **without losing or changing a single byte**. It only slices; it never rewrites. So even a giant 10-K costs us no text.

build_seed.py

Group into the seed. After the blind bots name drivers, this code gathers all their menus into one raw draft — no AI, just bookkeeping.

Takes in

menus/*.json (one file per bot)

Gives back

seed.json — the deduped driver records

Step by step

1. Read every bot's menu file.
2. Group candidates by their lowercased name (exact matches only).
3. Merge their evidence quotes; drop exact-duplicate quotes; list the distinct companies.
4. Cross-check: do the counts match what the bots reported? If not → **fail**.
5. Check every chunk produced a menu — no chunk silently skipped.

Real example

2,246 raw candidate names → 594 distinct drivers

It only merges *exact spelling* matches here. Real synonyms (`guest_count` vs `customer_transactions`) are left for the careful judges in Stage 2.

slice_seed.py

Batch a big seed. 594 drivers is too many for one AI to weigh carefully. So cut the seed into smaller piles.

Takes in

seed.json

Gives back

seed_batch_001.json, _002, ...

Step by step

1. Walk the records in name order (they're already sorted alphabetically).
2. Start a new batch when the current one hits **400 records** or **300,000 characters**.
3. A single record that's bigger than the limit goes alone — a record is never split in half.

Why it matters (and the small cost)

Each AI judge then only ever sees a small, careful pile. The trade-off: two synonyms with no shared words can land in different batches and miss each other — an **under**-merge (the safe direction). The repair tool (slide 55) sweeps those up later. If the seed is already small, it just makes one batch.

assemble_catalog.py

Write the catalog. This is "the writer." The AI only produced short decision lists; this code turns them into the actual file.

Takes in

seed.json + decisions.json (the AI's approved lists)

Gives back

catalog.json + approved.json

Step by step — the fixed 5-way rule, per driver

- | | |
|----------------------|---------------------------------|
| 1. gate said skip? | → skips list |
| 2. approved merge? | → point it at the survivor name |
| 3. approved rewrite? | → point it at the fixed name |
| 4. parked rewrite? | → unresolved list |
| 5. otherwise | → keep as its own driver |

Then it fills each survivor's `same_as_variants` (so trading reads the whole group in one hop).

Why it matters most

Because **code** writes the file from an approved list, the AI **cannot invent a merge** the skeptic never approved. This is what makes the old v2 mush structurally impossible. It also hash-checks that the bytes the AI typed match what the workflow intended.

validate_catalog.py

The hard gate. The last check. It re-inspects the structure and **stops the run** if anything is wrong. Zero judgment — pure checks.

Takes in

seed.json , catalog.json , approved.json

Gives back

PASS / FAIL + a signed receipt (validation_exit.json)

What it checks

- Every merge traces back to an approved link (it can't justify one → fail).
- Nothing lost or duplicated — every seed name accounted for exactly once.
- No invented names; the catalog's quotes match the seed's quotes exactly.
- It **recomputes** the "≥8 companies" list itself and demands the second-skeptic proof exists.

The receipt

It writes a signed note: "passed, and here's the catalog's fingerprint." Later steps refuse to use any catalog unless its receipt says passed **and** the file hasn't changed since. This catches "the check never actually ran" and "someone edited it afterward."

fold_catalogs.py

The merge math. The no-AI engine behind merging levels up. It has three small commands.

Its three jobs

part-a	tidy each child catalog, then group names across children. A name in 1 child → keep. A name in 2+ children → queue it for the AI (same spelling ≠ same meaning).
draw	build a fair sample of quotes for each queued name — smallest side first — so the AI sees the disagreement.
part-b	apply the AI's verdicts: SAME → merge, DIFFERENT → split, UNCLEAR → park. Write the parent seed.

Why it matters

It does the heavy, error-prone bookkeeping (union evidence, partition splits, sort, count) so the AI only makes the *meaning* calls. It refuses to fold a child whose validation receipt isn't passing-and-unchanged, and records every kept/dropped count so nothing vanishes silently.

menu_build.js

Runs the leaf. The conductor for Stage 1: turn one industry into a raw seed. It calls the worker tools in order and runs the blind bots.

```
resolve → fetch → chunk → [blind bots name drivers] → build_seed → record
```

Step by step

1. Billing guard: refuse to run if a pay-per-call API key is present.
2. Call `resolve` → tickers, `fetch` → text, `chunk` → pieces.
3. Spawn **one blind AI bot per chunk** to coin candidate driver names (this is the only AI work it does directly).
4. Call `build_seed` (code) to group them, then save a manifest of proof.

Run it like this

`menu_build.js` with `{ industry: "Restaurants" }` . If a long run dies, restart with `resume_run_id` → it re-runs only the missing bots, never re-pays for finished ones.

reconcile.js

The merge engine. The conductor for Stage 2: clean a messy seed into a real catalog. The same engine runs at **every** level later.

```
guard+slice → (dedup + gate, in parallel) → skeptic → assemble → validate
```

Step by step

1. Guard (billing) + slice the seed into safe batches.
2. Per batch: run two judges in parallel — **dedup** (find same-meaning pairs) and **gate** (admit / rewrite / skip).
3. **Skeptic** tries to break every proposed merge; a second skeptic checks merges touching ≥ 8 companies.
4. Hand the short approved lists to `assemble_catalog.py` (code writes the file).
5. Run `validate_catalog.py` — fail the run if anything is off.

Why it matters

This is the brain that decides what merges — but it never writes the file itself. It only produces a list of approved decisions for the code to apply. That separation is the whole safety story.

fold_catalogs.js

Merges levels up. The conductor that wraps the merge-math tool with the AI's same-name review — to fold several child catalogs into one parent.

part-a (code) → draw (code) → review (AI) → refute (AI) → part-b (code)

Step by step

1. Run `part-a` + `draw` (code): find the names that appear in 2+ children and build their evidence views.
2. For each clash, one AI **reviewer** says SAME / DIFFERENT / UNCLEAR.
3. A **skeptic** attacks every SAME; big ones get a second skeptic.
4. Run `part-b` (code): apply the verdicts and write the parent seed.

The output

A parent `seed.json` — which then goes through the exact same `reconcile.js` engine, one level up. That's how the tree climbs.

build_tree.js

The master conductor. Ties everything together and climbs the whole tree: company → industry → sector → global.

Its three modes

```
{ list: true }
```

Read-only: print the Sector → Industry → Ticker tree and fail loudly if any company has two parents. Never merges.

```
{ fold: {...} }
```

Do one merge: fold these children → reconcile → fold-check → repair → fold-check.

```
{ walk: {...} }
```

The full climb: fold every sector, then the global. Every step fail-closed.

The one rule it enforces

For each level it runs fold → reconcile → fold-check → repair → fold-check, and **one failure anywhere kills the whole run**. No level can be skipped, no "latest" guessing — you pass exact run IDs.

repair_duplicates.js

Catches missed merges. A second sweep for duplicates the batching split apart. Runs after each level.

suggest (code) → review (AI) → apply (code) → validate

Step by step

1. **Suggest** (code): find look-alike pairs by shared rare words, or by cheap similarity scores (embeddings). Suggestions only.
2. **Review** (AI): judge each pair SAME / DIFFERENT — same skeptic discipline as everywhere else.
3. **Apply** (code): add the approved links through the same writer, then validate.

Why it's safe

It only ever **adds** a link between two kept drivers — it never breaks an existing link and never touches the parked items. So it can only ever tidy up, never damage.

"Embeddings" = a cheap math score of how similar two names feel. Here it only points the AI at pairs to look at; it never decides a merge.

gate.js

The admission gate (called "G2"). The single bouncer for any new name — used both inside reconcile and, later, live.

Takes in candidate names (with their evidence) + the existing catalog

Gives back one verdict per name: reuse / admit / rewrite / skip

The one test it applies

"Is this a valid, reusable driver?" — judged from the **evidence**, not the bare name.

- **reuse** — it's the exact same cause as one already in the catalog
- **admit** — a new valid driver
- **rewrite** — right driver, just fix the wording
- **skip** — vague, or tied to one single event

It's deliberately "producer-agnostic" — it never tags a driver as "news vs fundamental." That's a job for whoever uses the catalog, not a property of the driver.

catalog_first.js

Reuse-first (called "G1"). The live mechanism for handling a brand-new event: name it yourself first, *then* reuse only if it's truly the same.

coin your OWN name (blind) → see the related catalog names → reuse if EXACT, else create / skip

Why "name it yourself first"?

If you saw the catalog up front, you'd lazily grab a near-match and slowly blur the meanings. Naming blind, then comparing, keeps everyone honest — the same anti-laziness trick as the blind leaf bots.

Status: not wired yet

This file still holds an **older** version of the flow (it shows the catalog up front). The new "name-it-first" rules — and exactly how related names are shown — are **still being decided**. The file header literally says: raise to the user before using. So it's the one piece here that is design-in-progress, not built.